

DSA STUDY BLUEPRINT SERIES

75. First Unique Character in String

Leetcode 387

Finding First Unique Character in String

Section 08 • C++ Core Data Structures & Algorithms

Core Concepts & Visual Blueprint

Theoretical models and memory architectures

Key Principles

- **Concept:** Read string once and count frequencies.
- Read string again and identify first char with count == 1.

Memory & Execution Blueprint

String: "leetcode" $\Rightarrow$ 'l' has count = 1 (Returns index 0)

C++ Implementation Reference

Standard code layout and syntax

```
int firstUniqChar(string s) {  
    vector count(26, 0);  
    for (char c : s) count[c - 'a']++;  
    for (int i = 0; i < s.length(); i++) {  
        if (count[s[i] - 'a'] == 1) return i;  
    }  
    return -1;  
}
```

Algorithmic Complexity & Best Practices

Performance evaluations and critical guidelines

Performance Table

Aspect / Case	Complexity
Time Complexity	$O(N)$ (Two Passes)
Space Complexity	$O(1)$ (Fixed count size)

Key Practices & Gotchas

- **Important:** Use queue buffers to process unique elements in dynamic string streams.
- Verify boundary conditions (e.g. empty inputs, single elements).
- Optimize dynamic memory allocations to prevent leaks.

Dry Run & Step-by-Step Execution

Trace analysis on a sample input case

Execution Walkthrough

Let's trace circular queue pointer wrapping when capacity = 3:

Operation	Queue data layout	Pointers [front, rear]	Size count
enqueue(10)	[10, _, _]	[0, 0]	1
enqueue(20)	[10, 20, _]	[0, 1]	2
dequeue()	[_ , 20, _]	[1, 1]	1 (wraps correctly)

Interview Variations & Optimization Pathways

Common follow-up problems and optimization strategies

Key Variations & Follow-up Questions

- **Wrap-around indices:** Modulo indexing updates circular buffer limits dynamically.
- **Deque mechanics:** Doubly ended queues allow fast $O(1)$ front and rear pushes and pops.
- **Related LeetCode Problems:** #239 Sliding Window Maximum, #622 Design Circular Queue, #225 Implement Stack using Queues.